

# Variadic Functions and Command-Line Arguments in C

## Aim

To implement C programs demonstrating a variadic function for weighted-average calculation and command-line argument processing for validation, sorting, and statistical calculations.

## Program 1: Weighted Average Using a Variadic Function

### Algorithm

1. Define `weightedAverage(int count, ...)` and initialize a `va_list`.
2. Read each value-weight pair using `va_arg()`.
3. Validate that all weights are non-negative.
4. Accumulate the weighted sum and total weight.
5. Call `va_end()` after processing the arguments.
6. If total weight is zero, display an error; otherwise return `weightedSum / totalWeight`.
7. Display the weighted average.

### C Program

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double value, weight;
    double weightedSum = 0.0, totalWeight = 0.0;
    int i;

    va_start(args, count);

    for (i = 0; i < count; i++)
    {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0)
        {
            printf("Error: Weight cannot be negative.\n");
            va_end(args);
            return -1.0;
        }

        weightedSum += value * weight;
        totalWeight += weight;
    }

    va_end(args);

    if (totalWeight == 0)
    {
        printf("Error: Total weight cannot be zero.\n");
        return -1.0;
    }

    return weightedSum / totalWeight;
}

int main()
{
    double result = weightedAverage(
        3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);

    if (result >= 0)
        printf("Weighted Average = %.2f\n", result);
    return 0;
}
```

```
}
```

### **Sample Calculation**

```
(80 x 0.3 + 75 x 0.2 + 90 x 0.5) / (0.3 + 0.2 + 0.5)
= (24 + 15 + 45) / 1.0
= 84.00
```

### **Test Cases - Program 1**

1. Standard input -> 84.00
2. Equal weights: 50, 70 -> 60.00
3. All weights zero -> Error
4. Negative weight -> Error

## **Program 2: Processing Integers Using Command-Line Arguments**

### **Algorithm**

1. Receive arguments through argc and argv.
2. Validate each argument; ignore and report non-integer values.
3. Store valid integers and terminate if none are supplied.
4. Sort valid integers in ascending order.
5. Display the sorted integers, minimum, and maximum.
6. Calculate and display the average and median.
7. Find and display the second-largest distinct value.

### **C Program**

```
#include <stdio.h>
#include <stdlib.h>

int isInteger(char str[])
{
    int i = 0;
    if (str[0] == '-' || str[0] == '+') i++;
    if (str[i] == '\0') return 0;

    while (str[i] != '\0')
    {
        if (str[i] < '0' || str[i] > '9') return 0;
        i++;
    }
    return 1;
}

void sortArray(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
}

double findAverage(int arr[], int n)
{
    int i;
    long long sum = 0;
    for (i = 0; i < n; i++) sum += arr[i];
    return (double)sum / n;
}

double findMedian(int arr[], int n)
```

```

{
    if (n % 2) return arr[n / 2];
    return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
}

int main(int argc, char *argv[])
{
    int arr[100], n = 0, i;
    int secondFound = 0, secondLargest;

    for (i = 1; i < argc; i++)
    {
        if (isInteger(argv[i]))
            arr[n++] = atoi(argv[i]);
        else
            printf("Invalid argument ignored: %s\n", argv[i]);
    }

    if (n == 0)
    {
        printf("No valid integers provided.\n");
        return 0;
    }

    sortArray(arr, n);

    printf("Valid integers in ascending order: ");
    for (i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    printf("Minimum: %d\n", arr[0]);
    printf("Maximum: %d\n", arr[n - 1]);
    printf("Average: %.2f\n", findAverage(arr, n));
    printf("Median: %.2f\n", findMedian(arr, n));

    for (i = n - 2; i >= 0; i--)
        if (arr[i] != arr[n - 1])
        {
            secondLargest = arr[i];
            secondFound = 1;
            break;
        }

    if (secondFound)
        printf("Second-largest distinct value: %d\n", secondLargest);
    else
        printf("Second-largest distinct value does not exist.\n");

    return 0;
}

```

## Compilation and Execution

```

gcc program.c -o program
./program 12 5 abc 8 12 20 3

```

## Expected Output

```

Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12

```

## Test Cases - Program 2

1. 12 5 abc 8 12 20 3 -> ignore abc; sorted: 3 5 8 12 12 20
2. 10 20 30 40 -> Min=10, Max=40, Average=25, Median=25
3. 5 5 5 -> no second-largest distinct value
4. -10 5 20 0 -> sorted: -10 0 5 20
5. abc xyz 10 -> ignore abc and xyz; valid integer=10
6. abc xyz -> no valid integers provided

## Result

Both C programs were successfully designed according to the given requirements. Program 1 uses a variadic function and `stdarg.h` to calculate a weighted average while validating weights and handling zero total weight. Program 2 uses command-line arguments, validation, sorting, and user-defined functions to determine the minimum, maximum, average, median, and second-largest distinct integer.